Purchasing Cranes

We can view the starting configuration as follows: the facility is already divided into *blocks* of lots (connected by existing cranes). Adding new cranes allows us to merge adjacent blocks. A delivery can be repaired iff every container lies in the same block as its lot.

Analyzing the structure and count of repairable deliveries

Suppose that we produced a final state in which the block sizes are $c_1, \ldots, c_k$. The reparable deliveries look as follows:

- $m = 1$: All deliveries are correct and thus repairable, all solutions are optimal (easy).
- $m = 2$: Incorrect deliveries are repairable only if both containers are in the same block. Thus, there are $\binom{n}{2}$ deliveries that are already correct and $\sum \binom{c_i}{2}$ incorrect repairable ones.
- $m = 3$: Here we can make our first non-trivial observation: for each incorrect but repairable delivery, **all repairing will still take place in a single block**. In addition to the $\binom{n}{3}$ already correct deliveries we now have, for each block separately:
  - $5\binom{c_i}{3}$ incorrect repairable deliveries with all three containers within the block
  - $(n - c_i)\binom{c_i}{2}$ incorrect repairable deliveries with two containers within the block (swapped) and one container outside the block (delivered correctly).

This analysis tells us an important property of our problem: **the goal we're optimizing is an additive function of individual block sizes**. That is, in all three cases we can formulate our optimization problem as follows: we are looking for the final state $c_1, \ldots, c_k$ for which some specific value $\sum_i W(c_i)$ is maximized.

The specific function W is $W(c_i) = 0$ for $m = 1$, $W(c_i) = \binom{c_i}{2}$ for $m = 2$, and $W(c_i) = 5\binom{c_i}{3} + (n - c_i)\binom{c_i}{2}$ for $m = 3$. Maximizing the sum of all $W(c_i)$ directly corresponds to maximizing the number of repairable deliveries, which is the same as maximizing the probability of a repairable delivery.

Merging blocks is always good

We can easily verify that whenever we have two adjacent blocks of sizes x and y, $W(x + y) \geq W(x) + W(y)$, hence it never makes sense to leave some merging budget unused. In other words, we just need to find the best state that can be obtained by making **exactly** b merges of adjacent blocks.

In the beginning we have $z + 1$ blocks, so in the end we will just have $r = z + 1 - b$ of them.

Another way of stating the same: we want to take the current block sizes $a_1, \ldots, a_{z+1}$ and split them into $z + 1 - b$ contiguous groups, maximizing the sum of W of each group size.

First polynomial solution using DP

Let P_i be the prefix sums of current block sizes a_i.

We can now solve our problem using the following DP:

- $dp[1][i] = W(P_i)$
- $dp[p][i] = \max_{p-1 \le j < i} \big(dp[p-1][j] + W(P_i - P_j) \big)$
- final answer: $dp[r][z+1]$

In words, $dp[p][i]$ is the value of the optimal sum of W obtainable by dividing the first i given blocks into exactly p groups. To calculate any of these values, we try all possibilities for the size of the last group.

At this point it is a good idea to implement this solution and use it to solve the subproblems P1 and P2 — both to make sure that we have the whole idea right (P1 is public), and to gain an estimate how fast it runs on the given test data.

A bit of time complexity analysis

Note that we don't need to evaluate all possible states: for a fixed p we are interested only in $i \ge p$ as leaving an empty group is never needed, and also only in $i \le \min(z+1, p+b)$ as for bigger i we don't have enough merges to go down from i groups to just p.

Thus, for each p there are only $O(b)$ actually reachable states, which gives us a total of $O(rb)$ states. For each of them we have $O(b)$ different values j to consider, which leads to the total time complexity $O(rb^2)$. To be precise, we should add a $+n$ in there to account for the initial reading and processing of the input.

Monotonicity

If we want a solution with a provably better asymptotic time complexity, we need to know and use something more about the problem we are solving. In particular, one such useful new observation in this problem is that our function W is always **convex**. I.e., both for $m = 2$ and $m = 3$ the second differences of the sequence of values of W are positive.

(Everywhere below we quietly ignore $m = 1$ where we get equality, because we know that for $m = 1$ all solutions work anyway. Thus, all our inequalities will be strict.)

Our goal now will be to understand how we can use this convexity to reduce the number of transitions we need to examine.

Let's write $F_j(i) = dp[p-1][j] + W(P_i - P_j)$ for the quality of the solution we get when we divide the first i blocks into p groups so that the first j of them form the first $p-1$ groups. Recall that for each p and i we are looking for the j for which the value $F_j(i)$ is maximized.

Let's now fix any two constants $j_1 < j_2$ and look at the following function of i: $g(i) = F_{j_2}(i) - F_{j_1}(i)$. We claim that for any choice of the js this g is always decreasing.

Indeed, let's compare $g(i)$ and $g(i+1)$.

- Each of them contains the same terms $dp[p-1][j_2] - dp[p-1][j_1]$, those can be ignored.
- In $g(i)$ we add $W(P_i - P_{j_2}) - W(P_i - P_{j_1})$.
- In $g(i+1)$ we add $W(P_{i+1} - P_{j_2}) - W(P_{i+1} - P_{j_1})$.
- Those are the two terms we want to compare.

- Write the first term as $W(x) - W(y)$ with $x = P_i - P_{j_2}$ and $y = P_i - P_{j_1}$. Note that $x < y$.
- With $a = P_{i+1} - P_i \geq 1$ (the size of block $i+1$), the second term is $W(x+a) - W(y+a)$.
- Comparing these two terms is the same as comparing $W(y+a) - W(y)$ and $W(x+a) - W(x)$.
- Each of these can be seen as the sum of a consecutive first differences of W.
- As the second differences of W are positive, the first differences are increasing.
- As $y > x$, each of the a terms that sum up to $W(y+a) - W(y)$ is bigger than the corresponding term in the other sum.
- Hence, $W(y+a) - W(y)$ is bigger than $W(x+a) - W(x)$.
- This implies that $g(i) > g(i+1)$.

What does this mean for our dynamic programming?

Imagine that we fixed the value p and we are trying to calculate the values $dp[p][i]$ for increasing values of i. For each i we are looking for the optimal candidate j. Consider any pair of candidates $j_1 < j_2$. When is j_2 the better of these two candidates? Precisely for those i for which $g(i)$ is positive. However, as we know that g is decreasing, we know that there is (at most) one phase transition: eventually g becomes negative, and from that point on for all larger i the smaller candidate j_1 is better.

Some additional intuition

- A candidate j means the p-th group starts at block $j+1$ and ends at block i, having size $P_i - P_j$.
- The smaller candidate j_1 means that the group starts earlier, so its group is already larger by $P_{j_2} - P_{j_1}$ lots.
- Because W is convex, the larger a group already is, the higher its marginal benefit when expanded further by increasing i.
- Therefore, as i grows, $F_{j_1}(i)$ grows faster than $F_{j_2}(i)$.
- Thus, the larger candidate j_2 (which produces smaller groups) can only be competitive for small i. Once i grows large enough, the convex bonus of the larger block created by j_1 dominates, and j_1 remains superior for all subsequent positions.

How do we use this to improve time complexity?

We will use a **candidate stack**.

Each row is processed left to right with a stack of pairs (candidate, last position it owns). The intervals partition the positions from i to the end of the row, and the top holds the largest candidate and the earliest interval. Every owner maximizes F among the candidates so far. At position i:

1. pop the top if its interval ends before i;
2. add candidate $j = i - 1$: while it beats the top at the last position of the top's interval, pop the top; then, if it beats the top at i, binary-search the last position where it still does and push it with that position (if the stack is empty, it owns the whole rest);
3. $dp[p][i]$ is given by the top of the stack;
4. the optimal candidate is stored as the parent of this state.

Why this works: the invariant is that every future position $\geq i$ is owned by a best candidate j so far, and the stack contains the candidates that still own something. Since g is decreasing, whenever a new candidate appears, it will beat any older candidate exactly on some (possibly empty) prefix of future positions.

Time complexity

Finding the initial blocks and prefix sums takes $O(n)$.

In row p of the DP, the index i ranges from p to $\min(z+1, p+b)$, so row p contains at most $b+1$ states. While we process the row, each candidate is pushed and popped at most once, and pushing triggers at most one binary search that runs in $O(\log n)$. Thus, processing row p takes time proportional to the number of states in that row times $O(\log n)$.

There are $r = (z-b+1)$ rows, and thus the entire running time is bounded from above by $O(rb \log n)$ = $O((z-b)b \log n)$.

In P3 the value b is small, and the value $z-b$ (i.e., the number of - characters in the output) is obviously at most n. In P4 the value $z-b$ is small and the value b is obviously at most n.

Equivalently, the smaller of the two values b and $z-b$ is exactly $\min(b, z-b)$ and the larger is at most n. Thus, another valid upper bound on the time complexity of our algorithm is $O(n \cdot \min(b, z-b) \cdot \log n)$.

Remark on test data composition

We tried our best to make sure that the problem cannot be solved by blindly applying *incorrect* optimizations. For instance, neither the divide-and-conquer optimization nor the "Aliens trick" (Lagrange multipliers) can be blindly applied to this problem. During test data generation we made sure that incorrect solutions based on such types of optimization fail sufficiently many different test cases.

For example, consider an input with block sizes $a = (12, 1, 4, 1)$ and $m = 2$. In this setting, the optimal counts of repairable deliveries for $b = 0..3$ are 72, 84, 136, 153. Neither this sequence nor its increments are concave, the jump from $b = 1$ to $b = 2$ is much bigger than the ones before and after.